FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Millares, Kurt Martin M.
Section:	BSCSSE - AN42
Professor:	Ms. Crisola Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import Counter
import math



# =============================================================
# 1. AGENT DEFINITION
# =============================================================

class Walker(ap.Agent):

    def setup(self):
        # Store the complete path of the agent
        self.path = []

    def step(self):
        x, y = self.position

        # -----------------------------------------------------
        # Movement rule
        # -----------------------------------------------------
```

```
        if self.model.p.movement_rule == "avoid":

            # Possible directions
            directions = [
                (1, 0), (-1, 0),
                (0, 1), (0, -1),
                (0, 0)
            ]

            random.shuffle(directions)

            # Try to move to an unoccupied position
            new_position = self.position

            for dx, dy in directions:
                new_x = max(
                    0,
                    min(self.model.p.grid_size[0] - 1, x + dx)
                )
                new_y = max(
                    0,
                    min(self.model.p.grid_size[1] - 1, y + dy)
                )

                candidate = (new_x, new_y)

                # Check whether another agent is already there
                occupied = any(
                    other is not self
                    and other.position == candidate
                    for other in self.model.agents
                )

                if not occupied:
                    new_position = candidate
                    break
```

```
            self.position = new_position

        elif self.model.p.movement_rule == "preferred":

            # Preferred direction: move right
            preferred_direction = self.model.p.preferred_direction

            # Sometimes make a random move
            if random.random() < 0.7:
                dx, dy = preferred_direction
            else:
                dx, dy = random.choice([
                    (1, 0), (-1, 0),
                    (0, 1), (0, -1)
                ])

            new_x = max(
                0,
                min(self.model.p.grid_size[0] - 1, x + dx)
            )
            new_y = max(
                0,
                min(self.model.p.grid_size[1] - 1, y + dy)
            )

            self.position = (new_x, new_y)

        else:
            # Original random-walk rule
            dx, dy = random.choice([
                (1, 0), (-1, 0),
                (0, 1), (0, -1)
            ])

            new_x = max(
                0,
                min(self.model.p.grid_size[0] - 1, x + dx)
```

```
            )
            new_y = max(
                0,
                min(self.model.p.grid_size[1] - 1, y + dy)
            )

            self.position = (new_x, new_y)

        # Record new position
        self.path.append(self.position)


# ============================================================
# 2. MODEL DEFINITION
# ============================================================

class RandomWalkModel(ap.Model):

    def setup(self):

        # Create agents
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            Walker
        )

        # Initialize random positions
        for agent in self.agents:

            agent.position = (
                random.randint(
                    0,
                    self.p.grid_size[0] - 1
                ),
                random.randint(
                    0,
```

```
                    self.p.grid_size[1] - 1
                )
            )

            # Record initial position
            agent.path.append(agent.position)

        # Create grid
        self.grid = ap.Grid(
            self,
            self.p.grid_size,
            torus=False
        )

        self.grid.add_agents(self.agents)

    def step(self):

        for agent in self.agents:
            agent.step()


# =============================================================
# 3. RUN SIMULATION
# =============================================================

num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size: "))
num_steps = int(input("Enter number of steps: "))

print("\nChoose movement rule:")
print("1 - Random")
print("2 - Avoid other agents")
print("3 - Preferred direction")

#Default value
preferred_direction = None
```

```
choice = input("Enter choice (1/2/3): ")

if choice == "2":
    movement_rule = "avoid"
elif choice == "3":
    movement_rule = "preferred"

    print("\nPreferred direction:")
    print("1 - Right")
    print("2 - Left")
    print("3 - Up")
    print("4 - Down")

    direction_choice = input("Choose direction: ")

    if direction_choice == "1":
        preferred_direction = (1, 0)
    elif direction_choice == "2":
        preferred_direction = (-1, 0)
    elif direction_choice == "3":
        preferred_direction = (0, 1)
    else:
        preferred_direction = (0, -1)
else:
    movement_rule = "random"


parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps,
    "movement_rule": movement_rule,
    "preferred_direction": preferred_direction
}
```

```
model = RandomWalkModel(parameters)
model.setup()


# ============================================================
# 4. ANIMATION WITH PATHS
# ============================================================

fig, ax = plt.subplots()

ax.set_xlim(0, grid_size)
ax.set_ylim(0, grid_size)

ax.set_xticks(range(grid_size + 1))
ax.set_yticks(range(grid_size + 1))

ax.grid(True)

# Scatter plot for agents
scat = ax.scatter([], [], s=150)

# One line for each agent
lines = []

for i in range(num_agents):

    line, = ax.plot([], [], linewidth=1)

    lines.append(line)


def update(frame):

    # Run one simulation step
    model.step()

    # Current positions
```

```
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]

    scat.set_offsets(list(zip(x, y)))

    # Draw path of every agent
    for i, agent in enumerate(model.agents):

        path_x = [position[0] for position in agent.path]
        path_y = [position[1] for position in agent.path]

        lines[i].set_data(path_x, path_y)

    return [scat] + lines


ani = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps,
    blit=True,
    repeat=False
)

plt.title(
    "Agent Movement - " +
    movement_rule.capitalize() +
    " Rule"
)

plt.show()


# ============================================================
# 5. FINAL POSITION ANALYSIS
# ============================================================
```

```
print("\n==============================")
print("FINAL POSITIONS")
print("==============================")

final_positions = []

for i, agent in enumerate(model.agents):

    position = agent.position

    final_positions.append(position)

    print(
        f"Agent {i + 1}: {position}"
    )



# ------------------------------------------------------------
# Count how many agents occupy each position
# ------------------------------------------------------------

position_counts = Counter(final_positions)

print("\n==============================")
print("POSITION DISTRIBUTION")
print("==============================")

for position, count in position_counts.items():

    print(
        f"Position {position}: "
        f"{count} agent(s)"
    )



# ------------------------------------------------------------
# Calculate average final position
```

```python
# ------------------------------------------------------------

average_x = sum(
    position[0]
    for position in final_positions
) / num_agents

average_y = sum(
    position[1]
    for position in final_positions
) / num_agents


print("\nAverage final position:")
print(
    f"({average_x:.2f}, {average_y:.2f})"
)


# ------------------------------------------------------------
# Calculate spatial spread
# ------------------------------------------------------------

distances = []

for x, y in final_positions:

    distance = math.sqrt(
        (x - average_x) ** 2 +
        (y - average_y) ** 2
    )

    distances.append(distance)


average_distance = sum(distances) / num_agents
```

```
print(
    f"Average distance from center: "
    f"{average_distance:.2f}"
)



# ============================================================
# 6. FINAL POSITION PLOT
# ============================================================

plt.figure()

x = [p[0] for p in final_positions]
y = [p[1] for p in final_positions]

plt.scatter(x, y, s=150)

plt.xlim(0, grid_size)
plt.ylim(0, grid_size)

plt.xticks(range(grid_size + 1))
plt.yticks(range(grid_size + 1))

plt.grid(True)

plt.xlabel("X position")
plt.ylabel("Y position")

plt.title("Final Distribution of Agents")

plt.show()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?

2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. If the number of agents is disproportionately high compared to the size of the simulation grid, the distribution of agents becomes increasingly crowded. In the collision-avoidance mode, the high density of agents results in fewer available cells for movement. Consequently, agents may have difficulty finding unoccupied cells and may remain stationary when no suitable cell is available. In the other movement modes, agents are more likely to occupy the same cells or cross each other's paths because collision avoidance is not enforced. Therefore, increasing the agent density has a greater effect on movement and spatial distribution when the grid size remains constant
2. Increasing the number of steps generally increases the spread of agents since said agents receive more opportunities to move to unoccupied cells.
3. No, the agents never move outside of the grid because the boundary restrictions prevent agents from moving to coordinates which are outside of the defined valid grid range.
4. This simulation can be used to model real-life phenomena by assigning independent variables to individual agents and then applying specific movement rules to determine their resulting behavior.